Dither App - Documentação Técnica e de Desenvolvimento

Arquitetura, processamento de imagem, persistência e empacotamento para Windows

Campo	Informação
Versão documentada	0.11.0 Beta
Plataforma e interface	Windows x64; interface em português do Brasil
Base documental	Código-fonte, testes e scripts do repositório
Data	14 de outubro de 2026

Este documento descreve o comportamento observado no repositório local. Os itens marcados como evolução são possibilidades futuras, não funcionalidades existentes.

As áreas de captura ao final foram deixadas vazias para receber imagens reais da aplicação.

Sumário

1. Visão geral e objetivos
2. Arquitetura e módulos
3. Fluxo de processamento
4. Importação múltipla, lote e presets
5. Pixel Scale e algoritmos
6. Transparência, workers e exportação
7. Persistência, testes e build
8. Estrutura, decisões, limitações e roadmap
9. Registro visual da execução

Escopo: os detalhes técnicos refletem o código 0.11.0 Beta inspecionado. Ideias futuras estão separadas das funcionalidades atuais.

1. Visão geral e objetivos

Dither App é um aplicativo desktop para Windows que aplica técnicas de dithering colorido e efeitos de pixel art a imagens raster. A janela apresenta original e resultado lado a lado, oferece controles de processamento e exporta PNG.

- Pré-visualizar e comparar o resultado com a imagem original.

- Controlar método, paleta, Pixel Scale, transparência e remoção de fundo.

- Guardar predefinições de processamento entre sessões.

- Importar várias imagens e exportá-las usando as mesmas configurações.

- Distribuir uma pasta portátil Windows e, opcionalmente, um instalador.

Entradas: PNG, JPG, JPEG, WebP e BMP. Saída: PNG em RGBA. Tecnologias observadas: Python, PySide6, Pillow e NumPy.

2. Arquitetura e módulos

Arquivo ou módulo	Responsabilidade observada
app.py	Cria QApplication, define identidade e versão, aplica estilo e abre MainWindow.
dither_app/main_window.py	Janela Qt, painéis, controles, algoritmos, workers e fluxos de prévia e exportação.
dither_app/settings.py	ProcessingSettings imutável e valores padrão da predefinição Normal.
dither_app/presets.py	Validação, serialização JSON e persistência dos presets de usuário.
dither_app/batch.py	Validação de extensões, thumbnails de 64x64 e nomes de saída sem colisão.
dither_app/image_io.py	Abertura com correção EXIF e helpers de caminho/nome PNG.
dither_app/resources.py e styles.py	Localização de recursos em execução Python/PyInstaller e estilos visuais.
dither_app/processing.py, workers.py, widgets.py	Módulos de fachada que reexportam implementações mantidas em main_window.py.
tests/	Testes unittest de processamento, presets, lote, versão e configuração do instalador.

Nota arquitetural: as implementações principais ainda estão concentradas em main_window.py nesta versão.

3. Fluxo de processamento

- A imagem é convertida para RGBA; o helper de I/O corrige a orientação EXIF.

- Se habilitada, a remoção de fundo atua primeiro sobre a imagem.

- Quando o método não é Nenhum e Pixel Scale é maior que 1x, a imagem é reduzida com BOX.

- O método selecionado processa a imagem; métodos de paleta derivam cores dos pixels visíveis.

- As opções de alfa determinam a transparência final segundo o método.

- Com Pixel Scale acima de 1x, o resultado é ampliado ao tamanho original usando NEAREST.

- A prévia atualiza a interface; cada exportação processa novamente a imagem original em resolução completa.

A função process_image_with_settings(image, settings, cancel_callback) coordena o fluxo. O resultado é separado da origem, permitindo alterar configurações sem degradar a imagem original.

4. Importação múltipla, lote e presets

A janela aceita seleção múltipla e arrastar e soltar. Caminhos duplicados e formatos não suportados são ignorados. A lista mostra nome, dimensões, formato e thumbnail. A seleção carrega a imagem nos painéis; adicionar ou limpar a lista não apaga os arquivos de origem.

Exportar todas captura um único ProcessingSettings no início e aplica as mesmas opções sequencialmente aos arquivos. Os nomes seguem nome_dithered.png e recebem sufixo numérico se já existirem. Falhas por arquivo são isoladas e contabilizadas.

ProcessingSettings reúne método, cores, limiar, intensidade e seed do ruído, Pixel Scale, opções de alfa e remoção de fundo. Normal é interno e protegido. O usuário pode salvar, atualizar, selecionar e excluir presets; a interface indica alterações não salvas.

5. Pixel Scale e algoritmos

Pixel Scale varia de 1x a 12x. Valores acima de 1 reduzem largura e altura aproximadamente pelo fator escolhido com BOX, processam a versão menor e restauram a dimensão com NEAREST. O controle não é aplicado quando o método é Nenhum.

Método	Implementação observada
Threshold	Calcula luminância 0.2126 R + 0.7152 G + 0.0722 B. Valores maiores ou iguais ao limiar viram branco; os demais, preto.
Floyd-Steinberg	Usa paleta adaptativa MEDIANCUT e quantização de paleta Pillow com difusão Floyd-Steinberg.
Atkinson	Percorre pixels por linha, escolhe a cor mais próxima e distribui o erro dividido por 8 aos seis vizinhos definidos.
Bayer 2x2, 4x4 e 8x8	Gera matriz ordenada recursiva, repete o mapa de limiares na imagem e ajusta RGB antes de consultar a LUT de cores.
Ruído	Adiciona ruído uniforme RGB com gerador NumPy e seed configurável antes do mapeamento para a paleta. Intensidade zero não gera ruído.
Nenhum	Mantém a cor e aplica as opções de alfa, sem dithering nem Pixel Scale.

A paleta adaptativa usa até 100.000 pixels visíveis na amostra. Bayer, Atkinson e Ruído usam uma LUT RGB de 5 bits por canal para acelerar a escolha da cor próxima.

6. Transparência, workers e exportação

Preservar transparência mantém o alfa de origem. Com Alfa rígido, alfa igual ou superior a 128 vira opaco e alfa menor vira transparente. Sem preservação, a saída fica opaca. Pixels totalmente transparentes têm RGB zerado.

A remoção por cor compara a distância RGB ao quadrado com 3 vezes a tolerância ao quadrado. No modo global, remove todas as correspondências não transparentes. No modo conectado, uma busca em quatro direções parte das bordas e remove somente pixels correspondentes alcançáveis.

QObject e QThread executam processamento/exportação em segundo plano. ImageProcessingWorker e BatchExportWorker enviam progresso, resultado, falha ou cancelamento por sinais Qt; os workers não alteram widgets diretamente. O cancelamento é cooperativo e suas verificações variam conforme o algoritmo. Alguns sliders usam debounce.

Exportar atual pede um destino PNG e captura as configurações atuais. Exportar todas pede uma pasta e guarda esse diretório em QSettings. As duas modalidades gravam primeiro em arquivo temporário e publicam o destino concluído por substituição atômica. A exportação em lote trata cada imagem separadamente.

7. Persistência, testes e build

PresetManager salva JSON com schema_version 1 no AppConfigLocation do Qt, fora da instalação. Usa arquivo temporário e substituição atômica. Nomes têm até 60 caracteres, não podem ser Normal e não podem duplicar sem diferenciar maiúsculas/minúsculas. A leitura limita valores aos intervalos da interface. QSettings também guarda preferências da aplicação, como o último diretório de exportação.

Testes existentes: test_processing verifica RGBA, dimensões, preservação da origem e remoção conectada/global; test_presets cobre round-trip, gravação, nomes e schema; test_batch verifica extensões, colisões, thumbnails e lista; test_version confere 0.11.0 Beta; test_installer valida AppId e nomes do setup. O comando documentado é python -m unittest discover -s tests -v.

Build Windows: BUILDING.md usa Python 3.13. requirements.txt fixa NumPy 2.5.1, Pillow 12.3.0 e PySide6 6.11.1 com seus pacotes associados. build.ps1 executa testes e PyInstaller para dist/DitherApp; distribua a pasta completa. DitherApp-OneFile.spec e build-onefile.ps1 definem uma variante de arquivo único.

Instalador: build-installer.ps1 chama a build onedir e o compilador Inno Setup 6 ou 7. O script installer/DitherApp.iss define AppId, atalhos e instalação no perfil local. Antes, gere assets/app_icon.ico a partir de assets/app_icon_source.png com tools/generate_icon.py. O instalador ainda não tem assinatura digital.

8. Estrutura, decisões, limitações e roadmap

Caminho	Conteúdo
dither_app/	Interface, processamento, settings, presets, lote, I/O, recursos e workers.
tests/	Testes unittest existentes.
assets/	Arte-fonte do ícone; o ICO gerado é ignorado pelo Git.
build_config/ e installer/	Metadados de versão, script Inno Setup e instruções.
tools/ e scripts *.ps1	Ferramentas auxiliares e automação de build.
docs/ e assets/screenshots/	Documentação técnica, checklist GitHub e espaço para capturas reais.

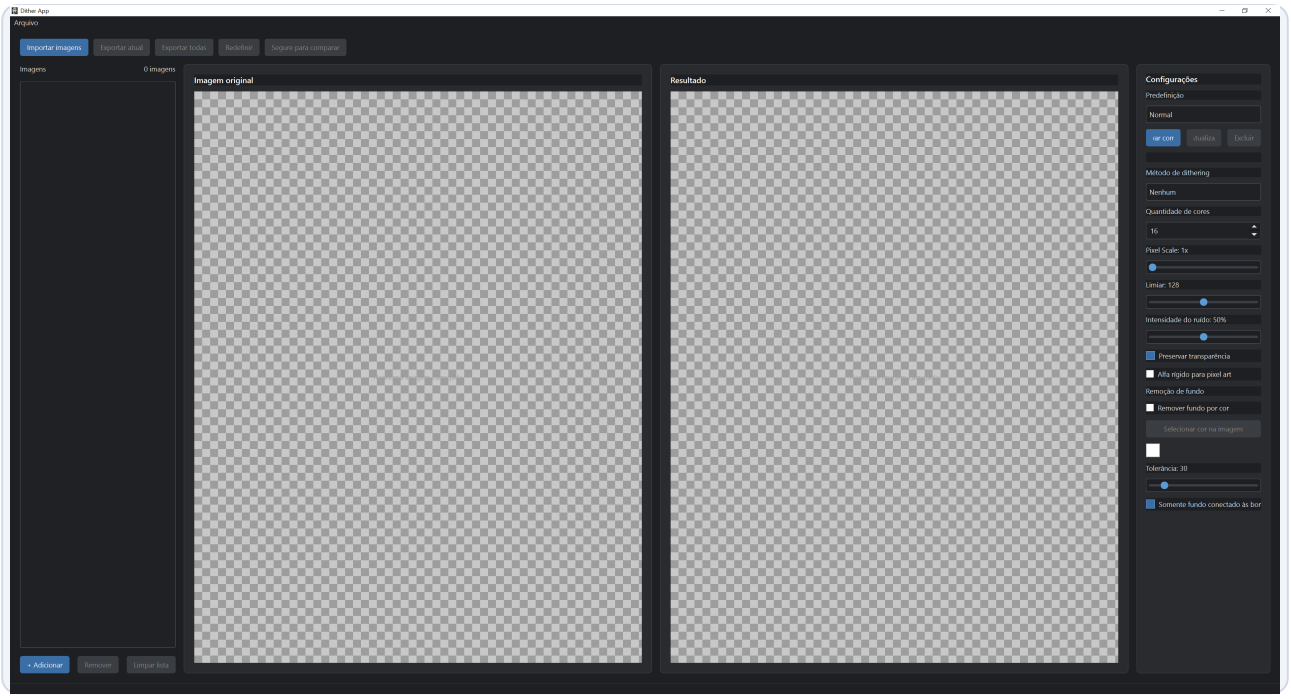
Decisões observadas: RGBA interno, PNG de saída, settings imutáveis por tarefa, presets fora da instalação e gravação temporária. **Limitações:** muitas implementações estão em main_window.py; a build depende do Windows; o instalador não está assinado; o ICO precisa ser recriado; não há licença open source nem capturas reais no repositório.

Evoluções planejadas, não implementadas: adicionar screenshots e guia de uso; automatizar CI e builds; melhorar o relato de falhas por arquivo; avaliar novos métodos e controles; definir licenciamento comercial e distribuição.

9. Registro visual da execução

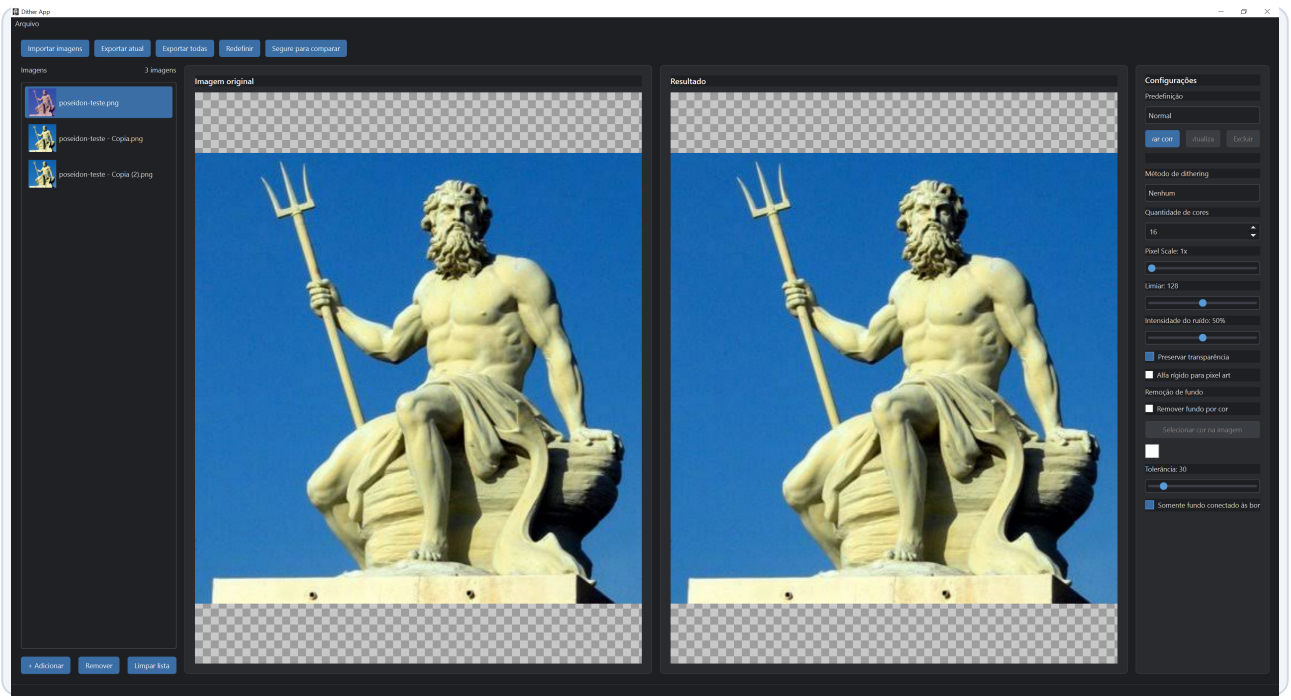
Capturas reais - interface e lista de imagens

Imagens fornecidas pelo usuário, capturadas durante a execução do Dither App.



01 | Janela principal sem imagens carregadas

`01-empty-main-window.png`

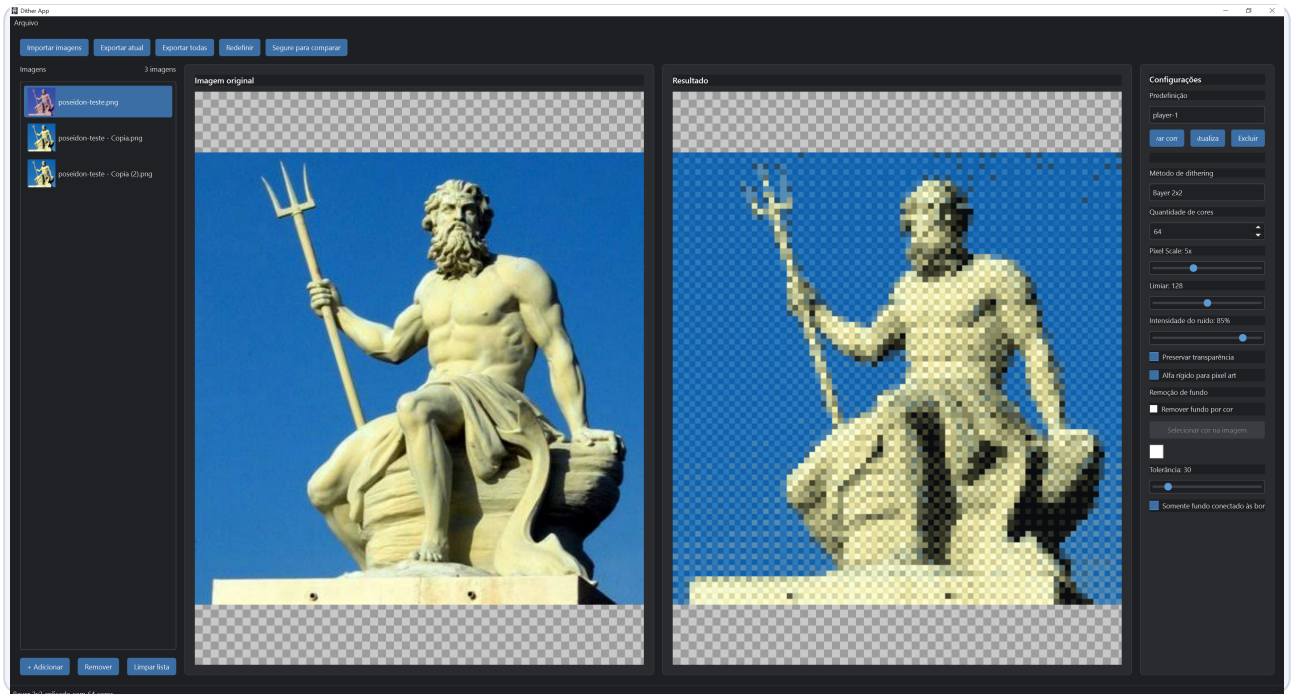


02 | Três imagens importadas e prévia lado a lado

`02-batch-preview.png`

Capturas reais - processamento e Pixel Scale

Exemplos visuais dos controles de dithering e da remoção de fundo.



03 | Bayer 2x2 com Pixel Scale 5x

`04-bayer-pixel-scale.png`

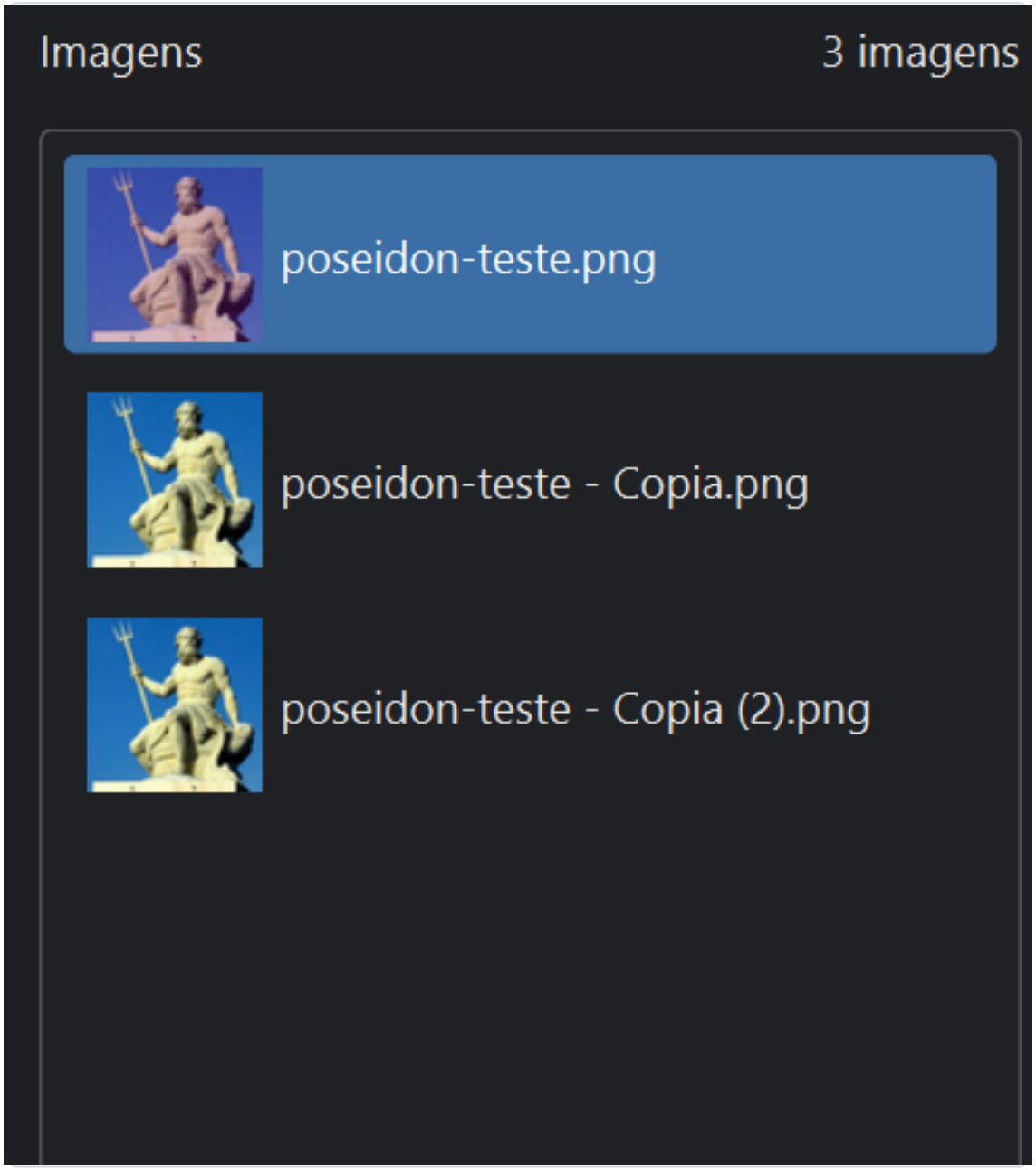


04 | Remoção de fundo e transparência

`05-background-removal.png`

Detalhe da lista de imagens

A captura mostra os arquivos adicionados à lista de processamento em lote.

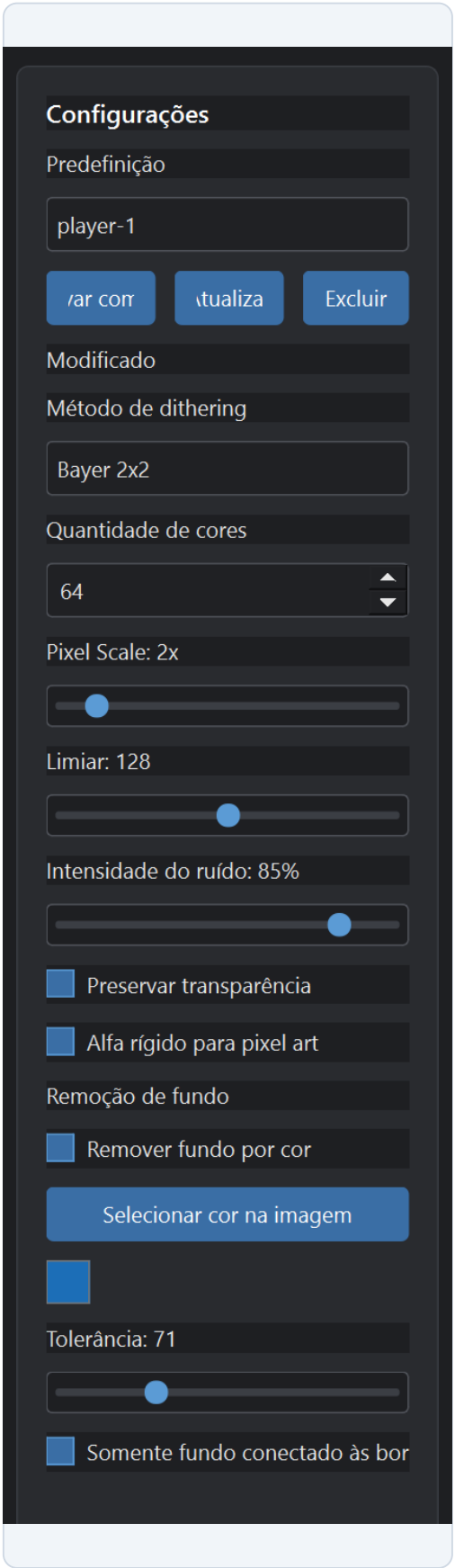


05 | Lista com três imagens e thumbnails

`03-image-list-detail.png`

Detalhe das configurações e preset

Recorte fornecido do painel lateral, incluindo método, cores, escala e remoção de fundo.



06 | Configurações do preset player-1

`06-settings-panel-detail.png`

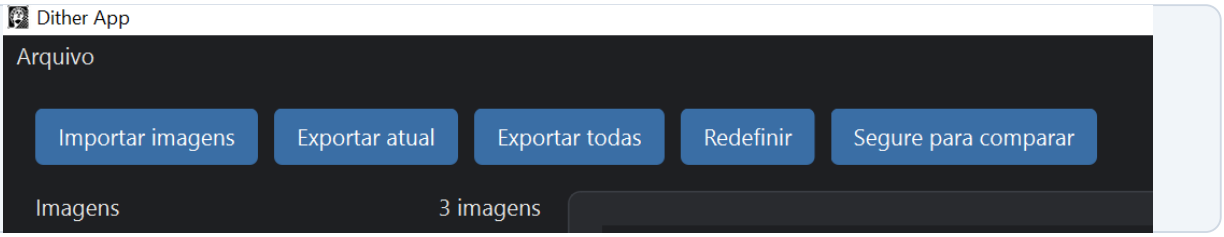
Detalhes da comparação e da interface

Recortes fornecidos pelo usuário para mostrar resultado, barra superior e estado do lote.



07 | Comparação original e resultado Bayer 2x2

`07-before-after-detail.png`



08 | Barra de ações e contador de imagens

`08-toolbar-detail.png`



09 | Controles da lista e mensagem de processamento

`09-batch-controls-status.png`